

## TECNOLOGÍAS DE LA INFORMACIÓN

### Desarrollo de Sistemas Informáticos

4 créditos



#### Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

| Titulaciones                                    | Semestre      |
|-------------------------------------------------|---------------|
| • <i>Tecnologías de la Información en Línea</i> | <i>Quinto</i> |

NOMBRES Y APELLIDOS: Ronnie Guillermo Tirira Morales

PARALELO: "A"

SEMESTRE: Quinto

ACTIVIDAD #1: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

PERIODO ABRIL 2026 – AGOSTO 2026



## Actividades a desarrollar en la Unidad 1

### Actividad # 1

Link Repositorio Github:

### INFORME

#### 1. Introducción:

En la actualidad, las organizaciones dependen significativamente de la infraestructura tecnológica para garantizar la continuidad de sus operaciones y servicios. Dentro de este entorno, los centros de datos desempeñan un papel fundamental, ya que concentran servidores, redes, sistemas de almacenamiento y aplicaciones críticas que requieren supervisión y mantenimiento constante. Debido a ello, la gestión adecuada de incidentes técnicos se ha convertido en una necesidad prioritaria para asegurar la disponibilidad y el correcto funcionamiento de los servicios informáticos.

#### 2. Objetivo:

Diseñar la arquitectura lógica de un sistema de gestión de incidentes técnicos (Help Desk) para infraestructura de servidores que permita aplicar correctamente patrones de diseño de software y flujos de trabajo profesionales (Gitflow), fortaleciendo la capacidad de análisis, abstracción y buenas prácticas en el desarrollo de soluciones escalables, reutilizables y mantenibles.

#### 3. Fundamentación teórica

El diseño de sistemas informáticos requiere la aplicación de metodologías y principios que permitan construir soluciones organizadas, reutilizables y adaptables a diferentes entornos tecnológicos. En este contexto, el desarrollo del sistema Help Desk se fundamenta en conceptos de ingeniería de software, diseño orientado a objetos y patrones de diseño, los cuales contribuyen a mejorar la calidad y mantenibilidad del software (Rodríguez, 2023).

El uso de diagramas UML facilita la representación gráfica de los procesos, actores y estructuras internas del sistema, permitiendo comprender de manera clara las interacciones y relaciones existentes entre los componentes. Además, UML se considera un estándar ampliamente utilizado en el análisis y diseño de software profesional.

Por otra parte, la aplicación de principios SOLID favorece la construcción de un sistema modular y flexible, reduciendo el acoplamiento entre componentes y facilitando futuras ampliaciones o modificaciones. Estos principios permiten mejorar la organización del código y promover buenas prácticas de desarrollo (Sánchez, 2023).



Asimismo, los patrones de diseño constituyen soluciones reutilizables para problemas comunes dentro de la ingeniería de software. En este proyecto se emplearán patrones como Singleton, Observer, Factory Method y MVC, debido a que ofrecen ventajas relacionadas con la centralización de procesos, automatización de notificaciones, creación flexible de objetos y separación de responsabilidades dentro de la arquitectura del sistema.

Finalmente, la implementación de GitHub y Gitflow como herramientas de control de versiones permitirá mantener un desarrollo organizado y estructurado, simulando entornos reales utilizados en proyectos profesionales. El uso de ramas, commits y fusión de cambios garantiza trazabilidad y control sobre la evolución del sistema, fortaleciendo las buenas prácticas de trabajo colaborativo y administración del software (Cánovas, 2021).

#### **4. Análisis del problema:**

En muchas organizaciones que administran infraestructura tecnológica, la gestión de incidentes técnicos suele realizarse de manera manual o mediante herramientas poco estructuradas, lo cual dificulta el seguimiento adecuado de los problemas reportados por los usuarios. Esta situación provoca retrasos en la atención de incidencias, pérdida de información relevante, desorganización en la asignación de técnicos y falta de control sobre el estado de las solicitudes.

Dentro de un Data Center, donde los servicios informáticos son críticos para el funcionamiento institucional, cualquier fallo relacionado con hardware, software o conectividad de red puede afectar significativamente las operaciones de la organización. Además, cuando no existe un sistema centralizado para registrar y monitorear incidentes, resulta complicado mantener trazabilidad sobre las actividades realizadas y los tiempos de solución.

Actualmente, uno de los principales problemas detectados es la ausencia de una plataforma que permita gestionar de manera eficiente los tickets de soporte técnico. Los usuarios reportan incidencias por diferentes medios, como llamadas telefónicas, mensajes o correos electrónicos, generando desorden y dificultad para priorizar problemas según su nivel de criticidad.

Asimismo, la asignación manual de técnicos puede ocasionar sobrecarga de trabajo en ciertos miembros del equipo de soporte, mientras que otros permanecen con



poca actividad. De igual manera, la falta de notificaciones automáticas retrasa la atención de incidentes urgentes y afecta la capacidad de respuesta del departamento de TI.

Frente a esta problemática, surge la necesidad de diseñar un sistema Help Desk que permita centralizar el registro de incidentes, automatizar la asignación de tareas, mantener control sobre el estado de los tickets y mejorar la comunicación entre usuarios, técnicos y administradores del sistema. La propuesta busca optimizar los procesos de soporte técnico mediante una arquitectura organizada y basada en buenas prácticas de desarrollo de software.

### **5. Actores del sistema:**

Para el funcionamiento del sistema Help Desk se identificaron tres actores principales, cada uno con responsabilidades específicas dentro del proceso de gestión de incidentes técnicos.

- a) Usuario final:** Corresponde a las personas que utilizan los servicios tecnológicos de la organización y que reportan problemas relacionados con hardware, software o red. Su principal función dentro del sistema es registrar incidentes y consultar el estado de sus solicitudes.

#### **Funciones principales:**

- Crear tickets de soporte.
- Describir incidentes técnicos.
- Consultar historial de tickets.
- Verificar el estado de las incidencias reportadas.

- b) Técnico de soporte:** Es el encargado de atender y resolver los incidentes asignados por el sistema o por el administrador. Posee conocimientos técnicos especializados dependiendo del tipo de problema reportado.

#### **Funciones principales:**

- Revisar tickets asignados.
- Actualizar el estado de incidencias.
- Registrar soluciones aplicadas.
- Resolver problemas técnicos.
- Recibir notificaciones de nuevos incidentes.



- c) **Administrador de sistemas:** Es el responsable de supervisar el funcionamiento general del sistema Help Desk y administrar los recursos relacionados con el soporte técnico.

**Funciones principales:**

- Gestionar usuarios y roles.
- Asignar técnicos especializados.
- Supervisar incidentes.
- Generar seguimiento de actividades.
- Administrar permisos y configuraciones.

**6. Requerimientos del sistema:**

- a) **Requerimientos funcionales:** Los requerimientos funcionales describen las acciones y servicios que el sistema debe realizar para satisfacer las necesidades de los usuarios.

- **RF1.-** El sistema debe permitir registrar tickets de soporte técnico.
- **RF2.-** El sistema debe clasificar los incidentes según su categoría: Hardware, Software, Red
- **RF3.-** El sistema debe asignar técnicos especializados según el tipo de incidente.
- **RF4.-** El sistema debe permitir actualizar el estado de los tickets.
- **RF5.-** El sistema debe generar notificaciones automáticas a los técnicos asignados.
- **RF6.-** El sistema debe almacenar el historial de incidentes y soluciones.
- **RF7.-** El administrador debe poder gestionar usuarios y roles del sistema.
- **RF8.-** El usuario final debe poder consultar el estado de sus solicitudes.
- **RF9.-** El sistema debe permitir registrar comentarios y soluciones técnicas.

- b) **Requerimientos no funcionales:** Los requerimientos no funcionales describen características de calidad y restricciones del sistema.

- **RNF1 Escalabilidad.** - El sistema debe permitir agregar nuevos tipos de incidentes y funcionalidades sin afectar su estructura principal.
- **RNF2 Mantenibilidad.** - La arquitectura debe facilitar futuras actualizaciones y correcciones del sistema.



- **RNF3 Seguridad.** - El sistema debe controlar el acceso mediante autenticación y roles de usuario.
- **RNF4 Disponibilidad.** - La plataforma debe encontrarse disponible para el registro y seguimiento de incidentes.
- **RNF5 Trazabilidad.** - El sistema debe mantener registro de cambios, actualizaciones y actividades realizadas.
- **RNF6 Modularidad.** - La estructura del software debe permitir reutilización y separación adecuada de componentes.

## **7. Diseño del sistema**

Por lo tanto, para el presente caso de estudio se propone el siguiente diseño del sistema:

### **Actores**

- Usuario Final
- Técnico de Soporte
- Administrador

### **Requerimientos funcionales**

- Crear tickets
- Asignar técnicos
- Resolver incidentes

### **Requerimientos no funcionales**

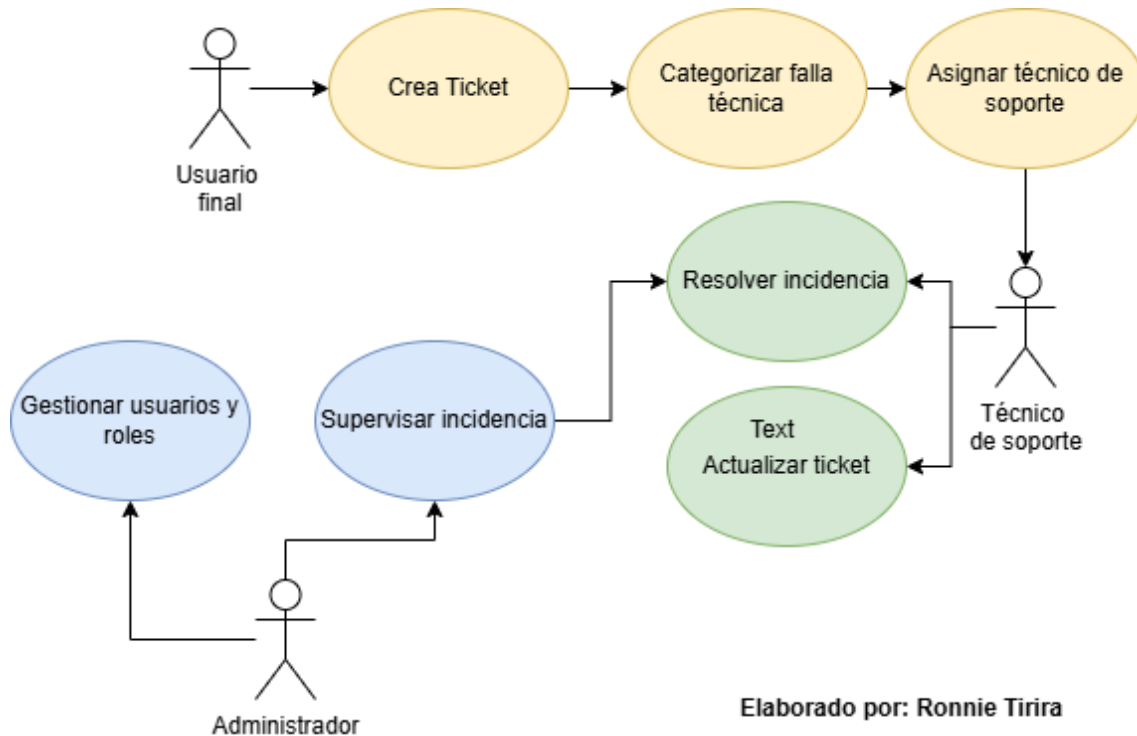
- Escalabilidad
- Seguridad
- Mantenibilidad

### **Diagramas**

- Casos de uso
- Clases
- Secuencia
- MVC

## 7.1. Diagramas

### Diagrama de casos de uso:



## 8. Conclusiones

El desarrollo de esta actividad permitió comprender la importancia del diseño de software en la construcción de sistemas organizados, escalables y mantenibles, aplicando principios de ingeniería de software orientados a buenas prácticas profesionales.

La utilización de diagramas UML permitió la representación visual de la arquitectura del sistema Help Desk, permitiendo identificar claramente las relaciones entre actores, procesos y componentes del sistema.

El uso de Gitflow y GitHub ayudó a la comprensión mediante el control de versiones y la organización del desarrollo colaborativo, simulando un entorno real utilizado en proyectos profesionales de software.



## Evidencias

*(Código de la ejecución de los ejercicios, enlace drive donde se encuentren los mismos y Capturas de pantalla claras y relevantes, donde evidencie el proceso realizado).*

## Práctica en Github

Primero, se realiza el proceso de registro y creación de cuenta en Github

### Sign up for GitHub

Email

rtirira8714@utm.edu.ec

[Use a different Google account](#)

Username\*

rtirira8714

Your Country/Region\*

Ecuador

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

Email preferences

☐ Receive occasional product updates and announcements

Create account >

github.com

Dashboard

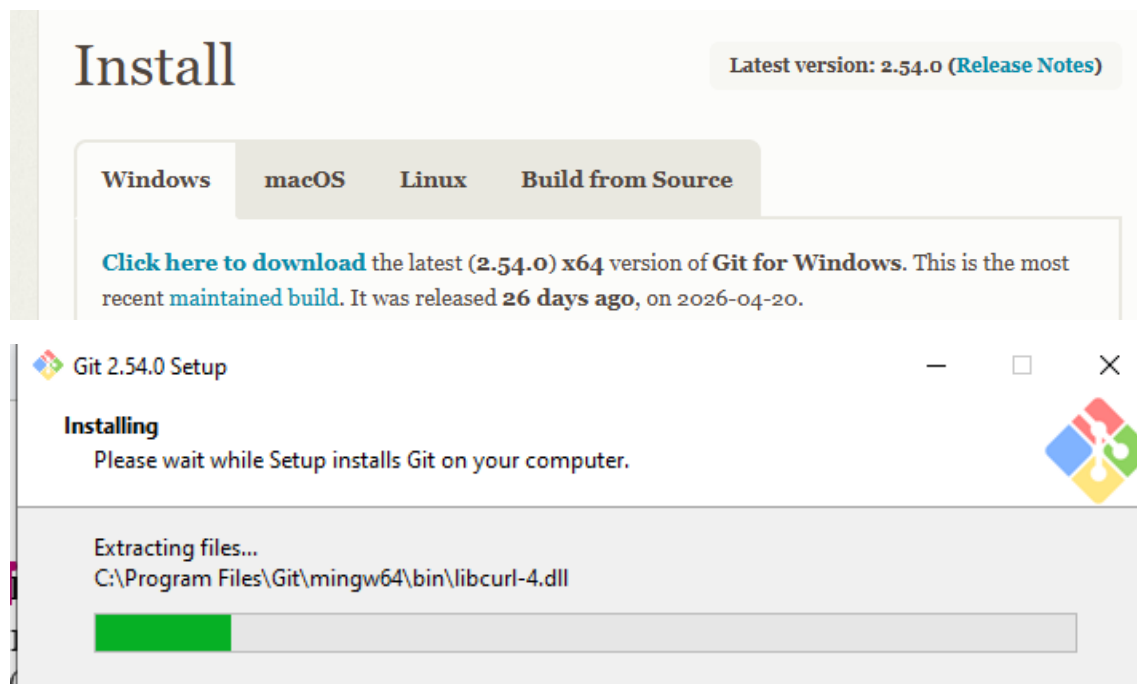
Create your first project

Ready to start building? Create a repository for a new idea or bring over an existing repository to keep contributing to it.

Create repository

[Import repository](#)

Luego, se instala Git para Windows.





```
MINGW64:/c/Users/Ronnie  
  
Ronnie@DESKTOP-KMB8692 MINGW64 ~  
$
```

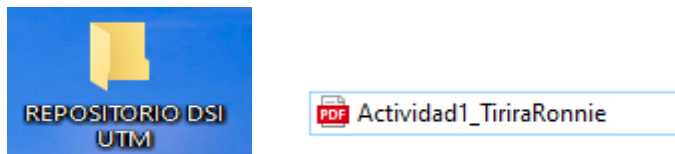
Después, se crea identificadores internos con los comandos Bash

Bash

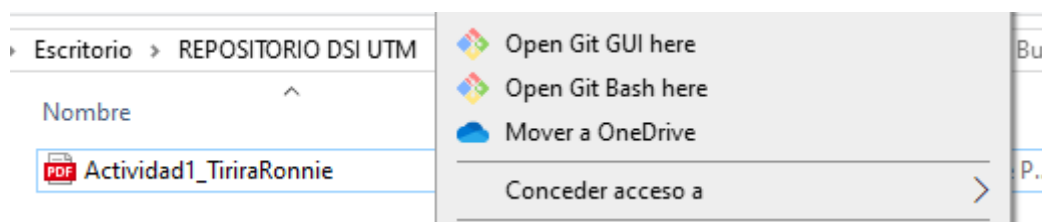
```
git config --global user.name "RONNIE"  
git config --global user.email "rtirira8714@utm.edu.ec"
```

```
MINGW64:/c/Users/Ronnie  
  
Ronnie@DESKTOP-KMB8692 MINGW64 ~  
$ git config --global user.name "RONNIE"  
  
Ronnie@DESKTOP-KMB8692 MINGW64 ~  
$ git config --global user.email "rtirira8714@utm.edu.ec"  
  
Ronnie@DESKTOP-KMB8692 MINGW64 ~  
$
```

A continuación, se crea la carpeta de repositorio y se agrega un archivo PDF



Luego, se inicia Git en la carpeta del repositorio creada.



Después, escribir el Bash git init.

```
MINGW64:/c/Users/Ronnie/Desktop/REPOSITORIO DSI UTM  
  
Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM  
$ git init  
Initialized empty Git repository in C:/Users/Ronnie/Desktop/REPOSITORIO DSI UTM/.git/  
  
Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM (master)  
$ |
```



Después, se realiza el primer commit de la versión 1 del pdf.

MINGW64:/c:/Users/Ronnie/Desktop/REPOSITORIO DSI UTM

```
Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM
$ git init
Initialized empty Git repository in C:/Users/Ronnie/Desktop/REPOSITORIO DSI UTM/.git/

Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM (master)
$ git add .

Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM (master)
$ git commit -m "commit inicial: subida de la primera version de la presentacionn"
[master (root-commit) b629efc] commit inicial: subida de la primera version de la presentacionn
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Actividad1_TiriraRonnie.pdf

Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM (master)
$
```

Luego, ir al perfil web y crear un repositorio en GitHub.

github.com/new

**Dueño \*** rtirira8714 / **Nombre del repositorio \*** DESARROLLOSOFTWAREUTM  
✓ DESARROLLOSOFTWAREUTM está disponible.

Los buenos nombres de repositorios son cortos y fáciles de recordar. ¿Qué tal si...? [enigma rediseñado](#)

**Descripción**  
DESARROLLO DE SISTEMAS INFORMATICOS  
0 / 350 caracteres

**2 Configuración**

**Seleccione la visibilidad \***  
Elige quién puede ver y realizar cambios en este repositorio. **Público**

**Agregar README**  
Los archivos README pueden utilizarse como descripciones más extensas. [Acerca de los archivos README](#) En ☒

**Agregar .gitignore**  
.gitignore le indica a git qué archivos no debe rastrear. [Acerca de los archivos ignorados](#) **No .gitignore**

**Agregar licencia**  
Las licencias explican cómo otros pueden usar tu código. [Acerca de las licencias](#) **Sin licencia**

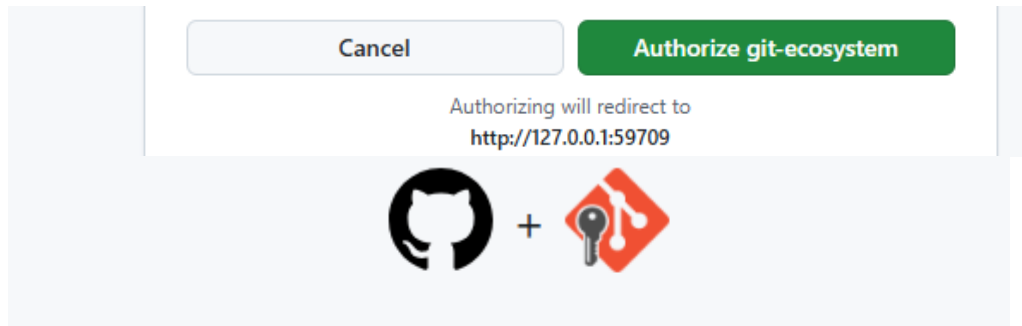
**Crear repositorio**

Posteriormente, se vincula y autoriza el acceso a la carpeta local con GitHub.

```
Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM (master)
$ git remote add origin https://github.com/rtirira8714/DESARROLLOSOFTWAREUTM.git

Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM (master)
$ git branch -M main

Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM (main)
$ git push -u origin main
```



## Authentication Succeeded

You may now close this tab and return to the application.

Después, se sincroniza para integrar los cambios remotos.

```
Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM (main)
$ git pull origin main --rebase
From https://github.com/rtirira8714/DESARROLLOSOFTWAREUTM
* branch      main      -> FETCH_HEAD
Successfully rebased and updated refs/heads/main.
```

```
Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 340.46 KiB | 21.28 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/rtirira8714/DESARROLLOSOFTWAREUTM.git
  2776147..99e5b70  main -> main
branch 'main' set up to track 'origin/main'.
```

Luego, se realiza la creación de ramas para la gestión del proyecto.

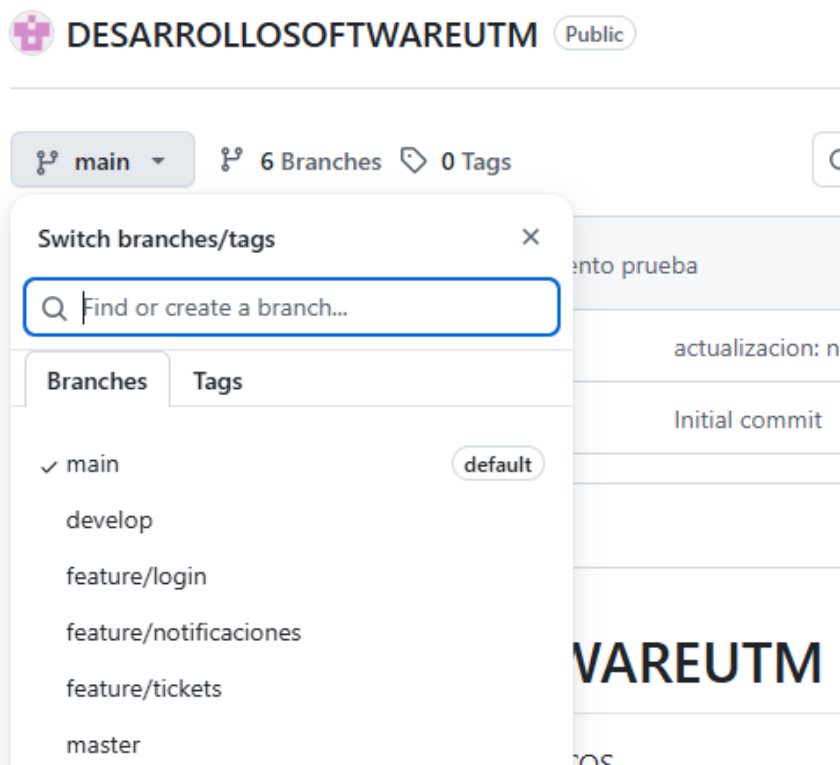
MINGW64:/c/Users/Ronnie/Desktop/REPOSITORIO DSI UTM

```
remote: Create a pull request for 'feature/tickets' on GitHub by visiting:
remote:   https://github.com/rtirira8714/DESARROLLOSOFTWAREUTM/pull/new/feature/tickets
remote:
To https://github.com/rtirira8714/DESARROLLOSOFTWAREUTM.git
* [new branch]      feature/tickets -> feature/tickets
branch 'feature/tickets' set up to track 'origin/feature/tickets'.

Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM (feature/tickets)
$ git checkout -b feature/notificaciones
Switched to a new branch 'feature/notificaciones'

Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM (feature/notificaciones)
$ git push -u origin feature/notificaciones
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/notificaciones' on GitHub by visiting:
remote:   https://github.com/rtirira8714/DESARROLLOSOFTWAREUTM/pull/new/feature/notificaciones
remote:
To https://github.com/rtirira8714/DESARROLLOSOFTWAREUTM.git
* [new branch]      feature/notificaciones -> feature/notificaciones
branch 'feature/notificaciones' set up to track 'origin/feature/notificaciones'.

Ronnie@DESKTOP-KMB8692 MINGW64 ~/Desktop/REPOSITORIO DSI UTM (feature/notificaciones)
$ |
```



## Bibliografía

(Referencias bibliográficas y/o enlaces en Internet (normas APA) que se utilizaron para realizar la práctica.)

Cánovas, J. (2021). *A systematic mapping study of software development with Github*.

Obtenido de Editorial IEEE Access:

<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7887704>

Rodríguez, J. (2023). *Estudio sobre la implementación del software Help Desk*.

Obtenido de Editorial Scielo: <https://www.scielo.org.mx/pdf/prts/v8n14/2007-3607-prts-8-14-00003.pdf>

Sánchez, G. (2023). *Principios SOLID en la automatización de pruebas de software*.

Obtenido de Editorial Politécnica: <https://revistapolitecnicaags.upa.edu.mx/wp-content/uploads/2023/12/V2116-.pdf>